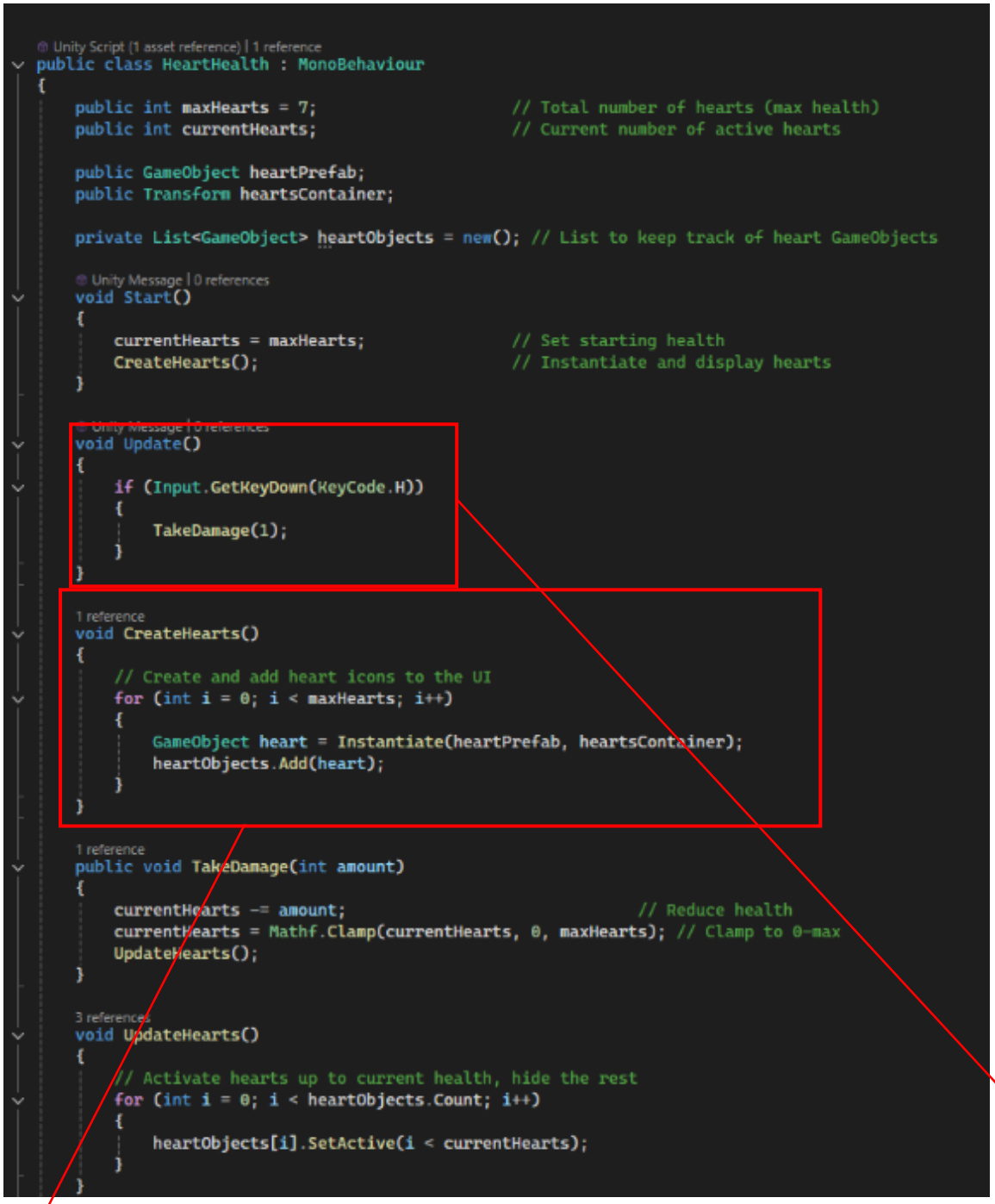
Health

19/04/25

Success criteria 10

After talking to the stakeholder I decided to add a health bar in the form of multiple hearts which can be seen in games such as Zelda and Minecraft as it will be more straightforward for when I create enemies the character loses one heart per hit, which therefore will save myself time.

I created a new separate code script in a folder I named variables for when I create stamina and damage variables. Below is the code I used, the comments explain what each part does.



```
Unity Script (1 asset reference) | 1 reference
public class HeartHealth : MonoBehaviour
{
    public int maxHearts = 7;                    // Total number of hearts (max health)
    public int currentHearts;                    // Current number of active hearts

    public GameObject heartPrefab;
    public Transform heartsContainer;

    private List<GameObject> heartObjects = new(); // List to keep track of heart GameObjects

    Unity Message | 0 references
    void Start()
    {
        currentHearts = maxHearts;               // Set starting health
        CreateHearts();                          // Instantiate and display hearts
    }

    Unity Message | 0 references
    void Update()
    {
        if (Input.GetKeyDown(KeyCode.H))
        {
            TakeDamage(1);
        }
    }

    1 reference
    void CreateHearts()
    {
        // Create and add heart icons to the UI
        for (int i = 0; i < maxHearts; i++)
        {
            GameObject heart = Instantiate(heartPrefab, heartsContainer);
            heartObjects.Add(heart);
        }
    }

    1 reference
    public void TakeDamage(int amount)
    {
        currentHearts -= amount;                             // Reduce health
        currentHearts = Mathf.Clamp(currentHearts, 0, maxHearts); // Clamp to 0-max
        UpdateHearts();
    }

    3 references
    void UpdateHearts()
    {
        // Activate hearts up to current health, hide the rest
        for (int i = 0; i < heartObjects.Count; i++)
        {
            heartObjects[i].SetActive(i < currentHearts);
        }
    }
```

This is for when I upgrade the amount of hearts my character has in the upgrade menu in the checkpoints, it adds a heart to the heart container which is an invisible box that contains all the hearts so no more can be added than possible and gives the code a location to add the hearts into.

This is simply so I can determine whether or not the losing a heart actually works for when I add enemies.

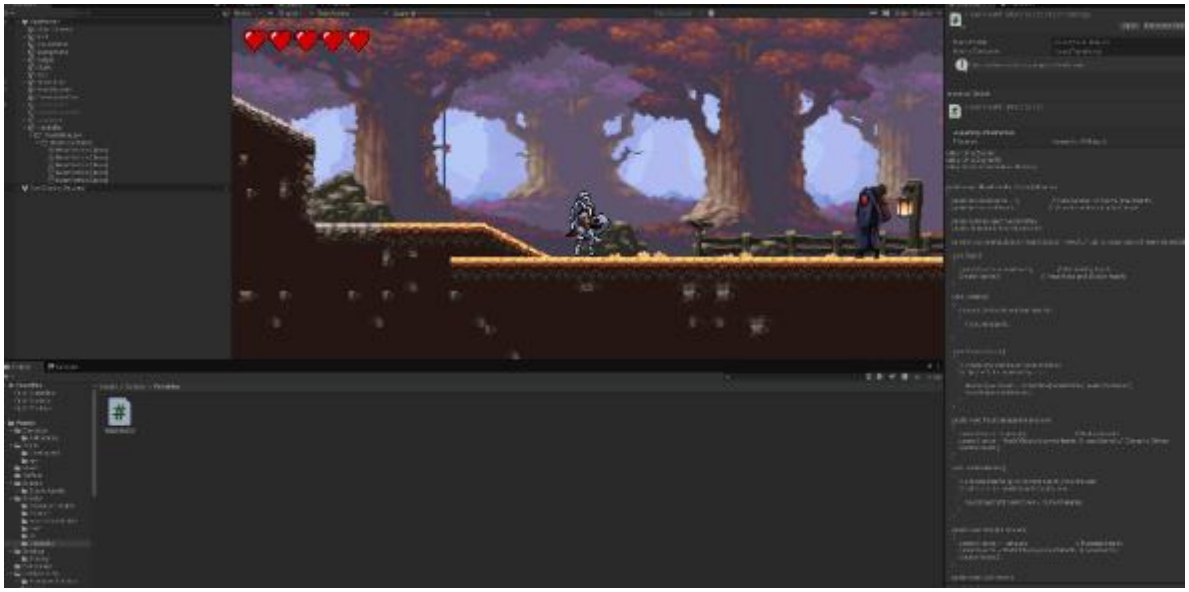
```
0 references
public void Heal(int amount)
{
    currentHearts += amount;                                    // Increase health
    currentHearts = Mathf.Clamp(currentHearts, 0, maxHearts);
    UpdateHearts();
}

1 reference
public void AddHeart()
{
    if (maxHearts < 7)
    {
        maxHearts++;
        currentHearts = maxHearts;

        GameObject heart = Instantiate(heartPrefab, heartsContainer);
        heartObjects.Add(heart);

        UpdateHearts();
    }
}
```

This limits the amount of hearts that can be added/ upgraded to which I have capped at 7 to not make the game too easy



This is what the game looks like in runtime.

Item	What is expected	Does it work?
Health bar	Hearts show in the top left corner and loses a heart when H is pressed	Yes



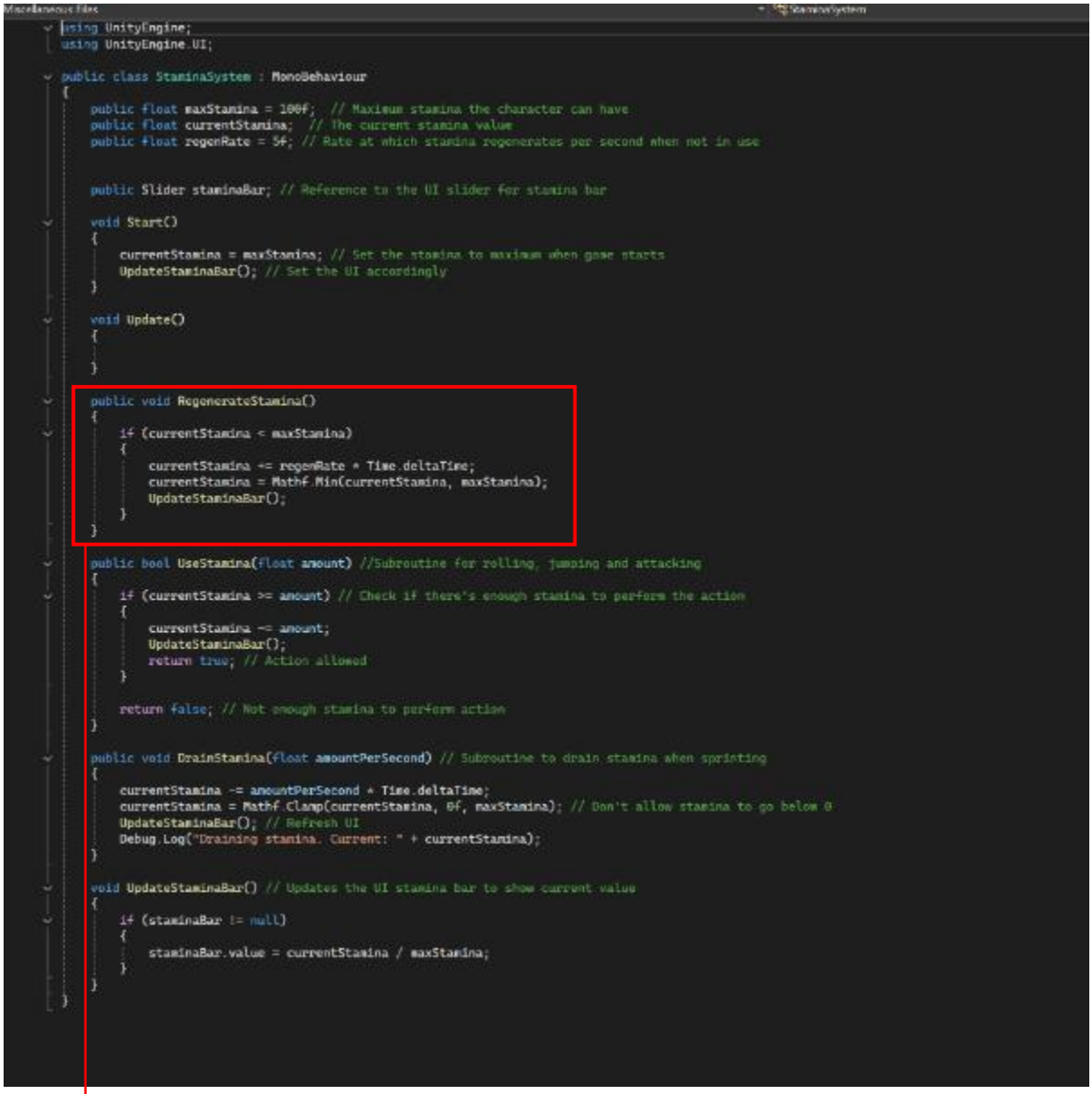
Stamina bar

29/04/25

Success criteria 11

Continuing from the success criteria, I added in a stamina bar to make the game more adaptive and strategic to play as it adds an extra layer of difficulty where you have to track how much stamina you use in order to allow yourself to still move.

To do this I created a new script aswell as setting up a new UI canvas and slider in the top left (under hearts) so the user can visualise how much stamina they are on inside unity. I've included comments to describe what each part does.



```
using UnityEngine;
using UnityEngine.UI;

public class StaminaSystem : MonoBehaviour
{
    public float maxStamina = 100f;  // Maximum stamina the character can have
    public float currentStamina;  // The current stamina value
    public float regenRate = 5f; // Rate at which stamina regenerates per second when not in use


    public Slider staminaBar; // Reference to the UI slider for stamina bar

    void Start()
    {
        currentStamina = maxStamina; // Set the stamina to maximum when game starts
        UpdateStaminaBar(); // Set the UI accordingly
    }

    void Update()
    {

    }

    public void RegenerateStamina()
    {
        if (currentStamina < maxStamina)
        {
            currentStamina += regenRate * Time.deltaTime;
            currentStamina = Mathf.Min(currentStamina, maxStamina);
            UpdateStaminaBar();
        }
    }

    public bool UseStamina(float amount) //Subroutine for rolling, jumping and attacking
    {
        if (currentStamina >= amount) // Check if there's enough stamina to perform the action
        {
            currentStamina -= amount;
            UpdateStaminaBar();
            return true; // Action allowed
        }

        return false; // Not enough stamina to perform action
    }

    public void DrainStamina(float amountPerSecond) // Subroutine to drain stamina when sprinting
    {
        currentStamina -= amountPerSecond * Time.deltaTime;
        currentStamina = Mathf.Clamp(currentStamina, 0f, maxStamina); // Don't allow stamina to go below 0
        UpdateStaminaBar(); // Refresh UI
        Debug.Log("Draining stamina. Current: " + currentStamina);
    }

    void UpdateStaminaBar() // Updates the UI stamina bar to show current value
    {
        if (staminaBar != null)
        {
            staminaBar.value = currentStamina / maxStamina;
        }
    }
}
```

This allows stamina to regenerate over time by multiplying the regen rate by 1 second (Time.DeltaTime) meaning that it will regenerate 5 stamina every second

After this I have to reference it in the characters movement script to make these subroutines have an affect on the character.

```
//Stamina
public StaminaSystem staminaSystem; // Reference the stamina script
public float sprintDrain = 10f; // How much stamina per second it costs to sprint
public float jumpCost = 15f; // How much stamina it costs to jump
public float rollCost = 15f; // How much stamina it costs to roll
private float lastStaminaUseTime = 0f; // When stamina was last used
public float staminaRegenDelay = 2f;   // Delay before regen starts in seconds
```

This is at the top of the script where I reference game objects and scripts and declare variables that will be used in the script.

```
//Run
if (Input.GetKey(KeyCode.LeftShift))
{
    if(staminaSystem.currentStamina > 0f) // Checks stamina is greater than 0 to ensure character has enough stamina to perform action
    {
        lastStaminaUseTime = Time.time;
        moveSpeed = 1f;
        transform.Translate(Vector2.right * moveSpeed * Time.deltaTime);
        animator.SetBool("isRunning", true);
        transform.localScale = new Vector2(0.75f, 0.75f);
        staminaSystem.DrainStamina(sprintDrain);
        lastStaminaUseTime = Time.time;
        isSprinting = true;

    }

}
```

This is for when the character runs, it adds an extra if statement around the code to make the character move to check if there is enough stamina for the character to perform the action. This is the same for the jump and roll but uses a different line of code that can be seen below due to sprinting occurring while the key is held down and the jump and roll being when the key is clicked once.

```
if(staminaSystem.UseStamina(jumpCost)) // Checks stamina is greater than the stamina cost to jump
```

```
if(staminaSystem.UseStamina(rollCost)) // Checks stamina is greater than the stamina cost to roll
```



Review of the health and stamina bars

Black = Question Red = Answer

- What do you enjoy about the health and stamina bars?
  I think the way they are shown on the screen is perfect, just like many other soulslikes, I think my idea of the hearts adds its own level of uniqueness so that the game will be able to be differentiated from other games. The stamina bar works well and just how I think it should be, as it doesn't take up much of the screen and bodes well with the style of the game.
- What could I improve about the health and stamina bar?
  I think the way the health and stamina bars are now perfect, and I cannot see a way to improve them.

I am happy with how the stakeholder likes the health and stamina bars, I also don't see a way to improve them, so I think this is a well done milestone.

Interactions

NPC

18/03/2025

Success criteria 6

After deciding to add in the NPC, I wanted him to talk to the character and explain to the user what they must do in the game. I think this is a good addition and it will help add a small amount of story/ lore to my game.

In order to create the script, there was extra implement UI into the base game and refere

Detects the characters distance from the NPC and displays the message press F to interact if in range

```
using UnityEngine;
using TMPro;
using System.Collections;

public class CharacterInteraction : MonoBehaviour
{
    public GameObject npc; // Reference to the NPC GameObject
    public float interactionDistance = 2.0f; // Distance within which interaction is possible
    public TextMeshProUGUI interactionText; // Reference to the TextMeshProUGUI element
    public float displayDuration = 2.0f; // Duration to display each message
    public float letterDelay = 0.05f; // Delay between each letter

    private bool isInteracting = false; // Flag to check if interaction is ongoing

    void Update()
    {
        float distance = Vector3.Distance(transform.position, npc.transform.position);
        if (distance <= interactionDistance && !isInteracting)
        {
            interactionText.text = "Press F to interact";
            if (Input.GetKeyDown(KeyCode.F))
            {
                InteractWithNPC();
            }
        }
        else if (!isInteracting)
        {
            interactionText.text = ""; // Clear the text when out of range
        }
    }

    void InteractWithNPC()
    {
        // Code for interaction with the NPC
        Debug.Log("Interacting with NPC...");
        isInteracting = true; // Set the flag to true
        string[] messages = {
            "Hello fateful Harbinger of Chaos, I have waited centuries...",
            "I am here to unveil your destiny, as dearest Solaire entrusted me after claiming the Chosen Undead's flame...",
            "You must harness the final Bonfires flame to destroy this world for good, Solaire willed it so for his sun was not as it seemed...",
            "I and the Great Solaire of Astoras wish you the bestest of luck!"
        };
        StartCoroutine(DisplayMessagesCoroutine(messages));
    }

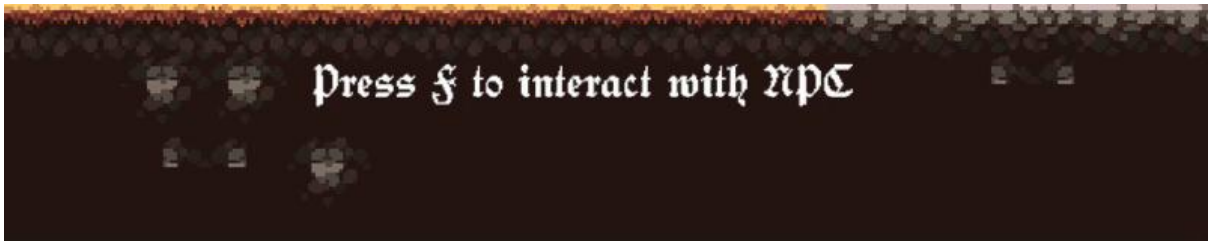
    IEnumerator DisplayMessagesCoroutine(string[] messages)
    {
        foreach (string message in messages)
        {
            yield return StartCoroutine(DisplayMessageLetterByLetter(message));
            yield return new WaitForSeconds(displayDuration);
        }
        interactionText.text = ""; // Clear the text after all messages are displayed
        isInteracting = false; // Reset the flag
    }

    IEnumerator DisplayMessageLetterByLetter(string message)
    {
        interactionText.text = "";
        foreach (char letter in message.ToCharArray())
        {
            interactionText.text += letter;
            yield return new WaitForSeconds(letterDelay);
        }
    }
}
```

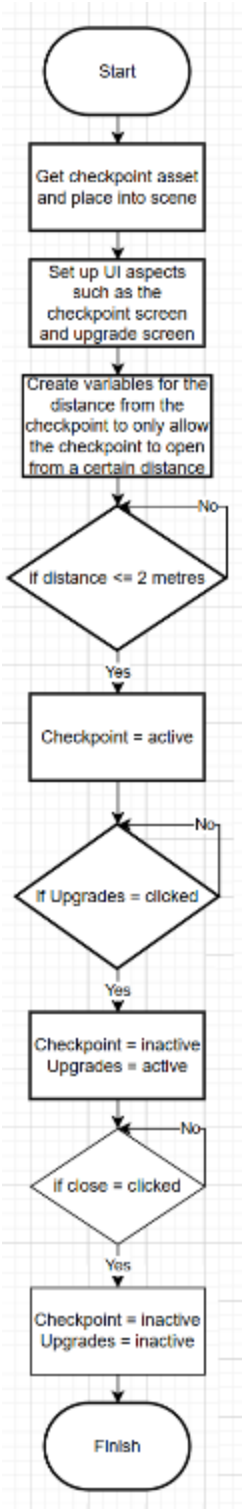
Makes the text show up letter by letter to add more retro pixel style to the game

Array of text that the NPC says/ shows up on the bottom of the screen, each comma is a new line that will type out after the old line

This displays the message from the array and makes the message disappear after time



Checkpoint(s)



I will follow this flowchart through changing my pseudocode to actual code and making any necessary changes.

26/03/2025

Success criteria 7, 8

To create working checkpoints that utilise UI and character Upgrades I decided to use the same script as the one for the Npc to create the idea of having one single script to handle all interactions that the character will do, this makes the code more organised and prevents me having to have loads of smaller scripts that I may need to call one script inside of another (which I don't know how to do) later in the creation. So this way it will make it much simpler and less tedious later on. (Utilising the computational thinking method: Thinking ahead)



First I created a simple checkpoint screen, I will later add to this. The cross in the top corner is a button that will send you back to the main game.

```
public GameObject checkpointone;
```

Declaring the gameobject of my checkpoint.

```
void Update()
{
    HandleInteractions();
}

void HandleInteractions()
{
    float distanceToNpc = Vector3.Distance(transform.position, npc.transform.position);
    float distanceToCheckpoint = Vector3.Distance(transform.position, checkpointone.transform.position);

    if (distanceToNpc <= interactionDistance && !isInteracting)
    {
        interactionText.text = "Press F to interact with NPC";
        if (Input.GetKeyDown(KeyCode.F))
        {
            InteractWithNPC();
        }
    }
    else if (distanceToCheckpoint <= interactionDistance && !isInteracting)
    {
        interactionText.text = "Press F to interact with Checkpoint";
        if (Input.GetKeyDown(KeyCode.F))
        {
            SceneManager.LoadScene(3);
        }
    }
    else if (!isInteracting)
    {
        interactionText.text = "";
    }
}
```

I had to add the HandleInteractions subroutine as when I previously wrote out the code it would get confused with distances and what gameobject it was accessing so it would only work for the checkpoint and not the npc. The handle interaction subroutine allows the code to understand what gameobject I am attempting to access at certain points.

Item	What is expected to happen?	Does it work?
Npc Interaction	Says press f to interact with NPC and when I do it writes out the sentences seen in the **NPC** part.	Yes
Checkpoint One Interaction	Says press f to interact with Checkpoint one and when I do it sends me to the checkpoint scene	Yes

After this I added the checkpoint UI, it took me quite a while to decide whether to have the UI on a separate scene, or simply add it into the main scene however, this comes with many difficulties and can cause a lot of crowding on the **Unity** game manager scene which can be annoying to work around. However, taking this all in I think it still is easier to add it into the main scene as with multiple scenes it can become tedious referencing them multiple times and I would have to reference separate scripts into others which I am not experienced in so overall in order to save time I chose to put it onto the main scene.

```
//CHECKPOINT UI
void CheckpointScreen()
{
    interactionText.text = "";
    Checkpoint.SetActive(true); // Make the Checkpoint screen show

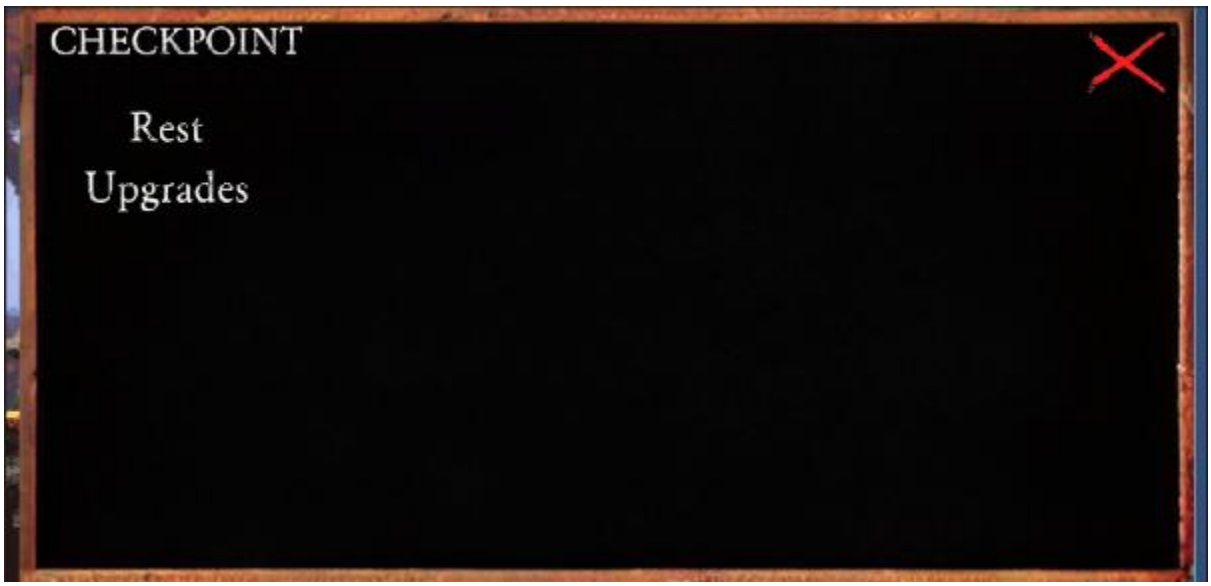
}

public void CheckpointExitButton()
{
    Checkpoint.SetActive(false);
}

public void UpgradesScreen()
{
    Upgrades.SetActive(true);
    Checkpoint.SetActive(false);
}

//UPGRADE UI
public void UpgradeExitButton()
{
    Upgrades.SetActive(false);
    Checkpoint.SetActive(true);
}
```

I added this simple code to the interaction script containing the open the checkpoint screen, exit the checkpoint screen, open the upgrades screen through the checkpoint screen and exit the upgrades to go back to the checkpoint one. This is useful as it means I can alter different parts of the code and upgrades to suit me, for example adding a health increase on the upgrades screen at a later date.



This is the checkpoint screen. I will add in when the rest button is clicked it will restore all health and flasks.

Success criteria 12



This is the upgrades screen which I can access after pressing Upgrades.

The X is used to exit the screen you are on which can be seen in the code above.

Health Increase

28/04/25

Adding a health upgrade is not that difficult as I have already added the necessary requirements in the health script in order to simply add a heart when clicked. I had to add few lines of code that can be seen below as well as reference the health script at the top of the checkpoint script as well as reference it in **Unity** so it knows what script to go into.

```
24
        0 references
25      public void IncreaseHealth()
26      {
27          if (heartHealth.currentHearts < 7)
28          {
29              heartHealth.AddHeart();
30
31          }
32      }
33
34  }
35
```

Stamina Increase

1/05/25

Adding in a Stamina upgrade is generally the same as the health bar as I want to increase the strategy and decision making of the game to make the player choose between certain upgrades.

```
public void IncreaseMaxStamina(float amount)
{
    maxStamina += amount; // Increases by set amount
    currentStamina = maxStamina; // Refill stamina on upgrade
    UpdateStaminaBar(); // Refreshes UI
}
```

I added this into the stamina script to allow me to reference it in the interaction/ checkpoint script and upgrade the stamina by a certain amount.

```
public void IncreaseStamina()
{
    if(staminaSystem != null)
    {
        staminaSystem.IncreaseMaxStamina(20f); // References the IncreaseMaxStamina subroutine from the Stamina Script and increases it by 20
    }
}
```

Added this to the interaction script to increase the stamina by 20.

This is all works well, however I want the player to be able to visualise this increase and you cannot see it in the slider as it doesn't increase in length with the upgrade, to do this I had to alter parts of the stamina script to do this.

```
        {
            RectTransform barTransform = staminaBar.GetComponent<RectTransform>();
            if (barTransform != null)
            {
                // Increase width by 20 units
                barTransform.sizeDelta += new Vector2(20f, 0f);
            }
        }
    }
}
```

UNT0026: GetComponent allocates even if no component is found.
Show potential fixes (Alt+Enter or Ctrl+.)

This increases the slider length by 20 which I can alter and reference in the upgrade part of the script. However, this caused the stamina bar to increase to the left, which would go off the screen so the user could not see the bar decreasing once it leaves the screen. To fix this I simply had to change the X-axis pivot anchor to zero so it cannot increase in the left, only the right.

Multiple checkpoints

2/05/25 - 11/05/25

After finishing off the stamina parts, I quickly realised that I would have to have multiple checkpoints, and this would not currently work with the code I had used.

Item	What is expected?	Does it work?
First checkpoint	Shows press f and when clicked the checkpoint screen appears	Yes
Second checkpoint	Shows press f and when clicked the checkpoint screen appears	No

I spent extremely long trying to come up with a solution for this as it deemed quite a difficult task to accomplish, however a lot of the time was spent with me searching the internet for solutions (**AI: ChatGPT** and **Copilot**, **How to implement checkpoints - Unity Engine - Unity Discussions**, **2d - In unity how can I make a checkpoint system - Game Development Stack Exchange**, **Unity-CheckPoints/README.md at master · santiandrade/Unity-CheckPoints · GitHub**) and piecing together many different parts of code for it to work. I first started off by changing the reference to the checkpoint game object into an array so inside of unity I could allocate multiple checkpoints to a single variable which I could put into the code so all the checkpoints will do exactly the same thing. I then had to change the checkpoint subroutine in handle interactions from an if to a foreach statement (foreach runs through each item in the array till it matches what is specified) to work out what checkpoint was the character close to and check the isInteracting flag is not currently in use, set it to true when f is pressed and reset it after I close the tab. I had to repeatedly use the debug log to verify this was working as for a long time **Unity** was struggling to call the code correctly as I intended it to.

```
1 reference
void HandleInteractions()
{
    bool isInteractingWithNpc = false;
    bool isInteractingWithCheckpoint = false;

    // NPC interaction
    float distanceToNpc = Vector3.Distance(transform.position, npc.transform.position);
    if (distanceToNpc <= interactionDistance && !isInteracting)
    {
        interactionText.text = "Press F to interact with NPC";
        isInteractingWithNpc = true;

        if (Input.GetKeyDown(KeyCode.F))
        {
            InteractWithNPC();
            isInteracting = true;  // Ensure flag is set when interacting with NPC
        }
    }

    // Checkpoint interaction
    foreach (GameObject checkpoint in checkpoints)
    {
        float distanceToCheckpoint = Vector3.Distance(transform.position, checkpoint.transform.position);
        Debug.Log("Distance to checkpoint: " + checkpoint.name + " = " + distanceToCheckpoint);
        if (distanceToCheckpoint <= interactionDistance && !isInteracting)
        {
            interactionText.text = "Press F to interact with Checkpoint";
            isInteractingWithCheckpoint = true;
            if (Input.GetKeyDown(KeyCode.F))
            {
                Debug.Log("Interacting with checkpoint: " + checkpoint.name); // Debug check
                CheckpointScreen(checkpoint);
                isInteracting = true; // Set interacting flag when checkpoint is activated
            }
        }
    }
```

This is the code I added which I explained above

Item	What is expected to happen?	Does is work?
Npc interaction	Says ‘press f to interact’ and when f is clicked the npc speaks to you	Yes
First checkpoint	Says ‘press f to interact’ and when f is clicked the checkpoint opens	Yes
Second checkpoint	Does the same as first checkpoint however it does actually allow the interaction and the isInteracting flag resets correctly	Yes

Review of the interactions

Black = Question Red = Answer

- What do you enjoy about the interactions?
  I like all the aspects of the interactions, I think they all work and are good for the game, giving me more options to change how I play and gives the game a more interactive experience. The NPC provides a greater feeling of the world being alive, and the dialogue he says is motivating.
- What could be improved about the interactions?
  To improve the interactions, I think that the text from the NPC is too large and it means that it makes the words more difficult to read in succession.

After reviewing the stakeholder feedback, I think it is a good idea to make the text smaller as I can see how it would make the text difficult to read. In order to do this I can simply change the size of the textmeshpro (text) object in the inspector to be smaller, this makes the games ease of play better.

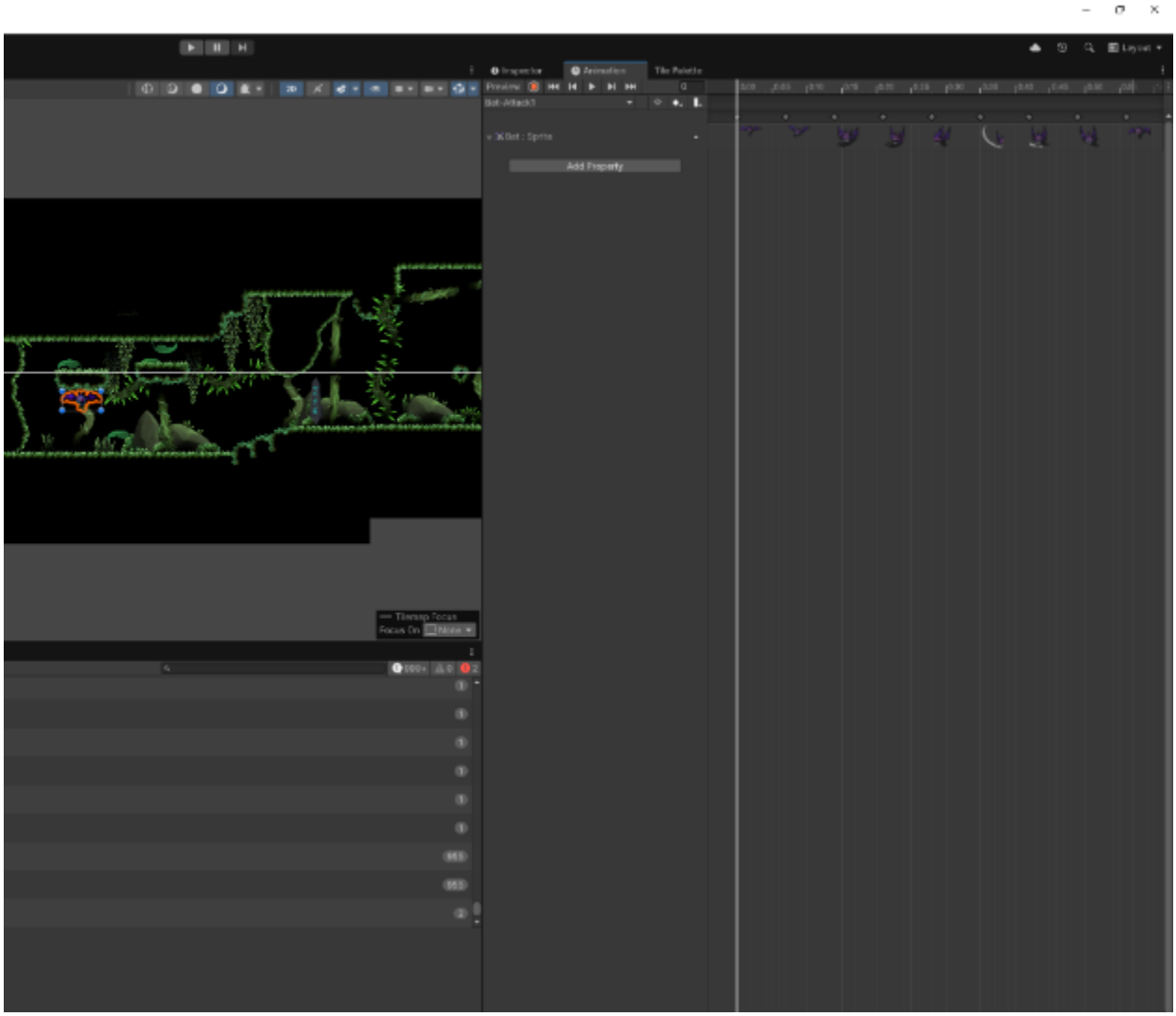
Enemies

Bat

24/07/25

The first enemy I will be adding is a bat, this will be in the first level and will be an easy enemy that will be killed with little problems and be a good introduction to the game.

I started by importing the bat asset and creating all the animations by using sprite pages that were provided.



I then linked them all together in the animator and after I have made a basic script I will create parameters so they all work together.

```
-CSharp                                                    EnemyBat
using UnityEngine;

Unity Script | 0 references
public class EnemyBat : MonoBehaviour
{
    public Transform player;
    public float chaseSpeed = 4f;
    public float detectionRange = 5f;

    private bool chasing = false;
    private SpriteRenderer spriteRenderer;

    Unity Message | 0 references
    void Start()
    {
        spriteRenderer = GetComponent<SpriteRenderer>();
    }

    Unity Message | 0 references
    void Update()
    {
        if (player == null) return;

        float distance = Vector2.Distance(transform.position, player.position);

        // Start chasing when player is close
        if (distance < detectionRange)
        {
            chasing = true;
        }
        // Stop chasing when player moves far away
        else if (chasing && distance > detectionRange * 1.5f)
        {
            chasing = false;
        }

        if (chasing) //Make the bat chase the player
        {
            Vector2 newPosition = Vector2.MoveTowards(
                transform.position,
                player.position,
                chaseSpeed * Time.deltaTime
            );
            transform.position = newPosition;

            // Flip bat to face the player
            if (spriteRenderer != null)
            {
                spriteRenderer.flipX = (player.position.x < transform.position.x);
            }
        }
    }
```

After creating the script, several errors occurred with the bat such as it becoming alerted to me way too early and the bat does not sit a certain distance away from me it just overlaps directly with the player.

I solved this by creating a stopping distance that the bat follows so it cannot overlap with me unless I intend it to do so.

```
1 reference
void ChasePlayer(float distance)
{
    if (distance > stopDistance)
    {
        transform.position = Vector2.MoveTowards(
            transform.position,
            player.position,
            chaseSpeed * Time.deltaTime
        );
    }

    FlipSprite(player.position.x < transform.position.x);
}
```

I then made the bat patrol a certain distance from where it starts to make the game feel more alive.

```
1 reference
void Patrol()
{
    float move = patrolSpeed * Time.deltaTime * (movingRight ? 1 : -1);
    transform.Translate(move, 0, 0);

    if (Mathf.Abs(transform.position.x - startPos.x) >= patrolRange)
    {
        movingRight = !movingRight;
        FlipSprite(movingRight);
    }
}
```

Next was making the animations work and making the bat attack the character. There was no real problems with implementing the animations as I simply made some parameters such as ‘isAttacking’ and put them into the code where necessary, referencing the animator before each line. There was various problems with getting the bat to actually damage the character though as before in the game I had multiple places referencing my health script which would in turn multiply the amount of hearts the character had in layers and would not remove some of them. Therefore I had to do multiple tests and use the inspector to determine which health script reference point I should actually use and had to replace each place where necessary, for example the characters. Then after solving this problem I simply referenced the subroutine that is in my health script to remove one of the players hearts after each hit in the bat script.

```
if (distance < attackDistance)
{
    animator.SetBool("isAttacking", true);

    if (Time.time - lastAttackTime > attackCooldown)
    {
        lastAttackTime = Time.time;
        if (playerHealth != null)
        {
            Debug.Log("Bat attacked player!");
            playerHealth.TakeDamage(1);
        }
    }
}
else
{
    animator.SetBool("isAttacking", false);
}
```

As seen if the bat is in the attacking distance the bat will play the attack animation and damage the player if the cooldown is not active.

Review of the enemies

Black = Question Red = Answer

- What do you enjoy about the enemy(s)?
  I think that the current bat enemy is a good start to the games enemies and shows good promise for the others to come. The way the bat wakes up when in proximity to the character is very nice and makes the game feel more immersive like the bat actually knows we are there promoting the feel of the games world being alive.
- What could be improved about the enemy(s)?
  I think that the bat could have smoother animations, as the bat sometimes jerks between the animations making the interaction feel less real.

After reviewing the stakeholder feedback, I agree with the stakeholder on the bat's animations. In order to make the animations smoother I can simply change the exit times of the switches in animation to 0, just like I did with the character's movement.

Reflection

I think the development went well, the stakeholder feedback was useful throughout and it provided me with good ways to make my game better. The feedback also showed that I was producing my game exactly how I wanted it to be, and that my game has good potential to be what I think is a great soulslike game. I moved through the milestones correctly and efficiently, allowing me to get most of the hardest parts of my development out of the way, so if I was to continue on with the game I would have a lighter time with the rest of the project compared to some of the more difficult and awkward parts of the project I have done.

Evaluation

Testing

Testing of entire game/ success criteria

No.	Testing	Input	Expected outcome	Actual outcome
1.	Click each button, click around the screen and press keyboard to see if anything will happen	Left-click on mouse	All buttons do their designated jobs (Start game, Exit game)	Works as expected

	(Destructive/ stress testing)			
2.	Click the button and music should turn on/ off. Press keys/ other inputs to see if anything will happen. (Destructive/ stress testing)	Left-click on mouse	Music plays and when button is clicked once it will turn off, when clicked again it will turn back on.	Works as expected
3.	Loading screen appears for an amount of time and then game plays.	N/A	Loading screen appears for an amount of time and then game plays.	Works as expected
4.	Levels are present in the game for the character to traverse. Find areas where character could fall through map/ get stuck.	N/A	Levels are present in the game for the character to traverse.	First two levels work as expected, Last two N/A
5.	Test each key input to ensure it works. Mash keys to try and break the movement (Destructive/ stress testing). Click the screen to see if anything happens.	Q, E, A, D, Z, C, Shift and Ctrl keys	All movement works correctly	Works as expected
6.	Press interact key when inside and outside of range to ensure detection and interaction works.	F key	Prompt to interact within certain distance. NPC talks to you	Works as expected
7.	Press interact key when inside and outside of range to ensure detection	F key	Prompt to interact within certain distance.	Works as expected

	and interaction works.			
8.	Ensure each screen opens when clicked. Click around the screen and press keys to see if anything else happens.	F key	Checkpoint screen opens and allows you to navigate around the checkpoint UI	Works as expected
9.	Ensure enemies detect you and then attack	N/A	Enemies detect you and then attack	Bat works correctly. Bosses and other enemies N/A
10.	Health bar is present and when attacked a health is lost.	N/A	Health bar is present and when attacked a health is lost.	Works as expected
11.	Stamina bar is present and when character moves stamina is lost, then regained over time	N/A	Stamina bar is present and when character moves stamina is lost, then regained over time	Works as expected
12.	Screens are present for each. Ensure when each one is clicked they do desired functions. Click around the screen and press keys to see if anything happens. Destructive/ stress testing.	Left-click mouse	Screens are present for each. Each screen/ button does desired function	Health and Stamina works as expected. Attack is N/A
13.	Item bar is present on the screen. Mash keyboard to see if it will break/ if anything happens	Numbers on keyboard	Item bar is present on screen and items can be used when numbered keys are pressed	N/A

	(Destructive/ stress testing).			
14.	Flames are present on the screen. Flames are lost once character is upgraded.	N/A	Flames are present on the screen. Flames are lost once character is upgraded.	N/A
15.	Enemies have multiple attacks and will rotate through them.	N/A	Enemies have multiple attacks and will rotate through them.	N/A
16.	When character rolls they are invincible to damage.	Z, C keys	When character rolls they are invincible to damage.	N/A
17.	When a certain key (Not decided) clicked it will regain certain amount of health. Click multiple keys to see if anything happens. (Destructive/ stress testing)	(Not decided) key	When a certain key (Not decided) clicked it will regain certain amount of health.	N/A
18.	Flask count present on screen with icon.	N/A	Flask count present on screen with icon.	N/A
19.	Flask count resets after resting at a checkpoint or respawning	N/A	Flask count resets after resting at a checkpoint or respawning	N/A
20.	Item present in game and when used it will upgrade flask amount. Press different keys on the item to see if anything happens.	Numbers on keyboard	Item present in game and when used it will upgrade flask amount	N/A

21.	When user dies a death screen appears.	N/A	When user dies a death screen appears.	N/A
22.	User respawns at the last visited checkpoint	N/A	User respawns at the last visited checkpoint	N/A
23.	Bosses are larger than enemies	N/A	Bosses are larger than enemies	N/A
24.	Boss health bar shows at the bottom of the screen and decreases when attacked after entering arena	N/A	Boss health bar shows at the bottom of the screen and decreases when attacked after entering arena	N/A
25.	When the user moves the screen follows with the character.	N/A	When the user moves the screen follows with the character.	Works as expected
26.	Different bosses add different status effects to the character when attacked.	N/A	Different bosses add different status effects to the character when attacked.	N/A
27.	Items that are used to nullify status effects are present. Press different keys on the item to see if anything happens.	Numbers on keyboard	Items that are used to nullify status effects are present.	N/A
28.	Different music for each level is present.	N/A	Different music for each level is present.	Works as expected
29.	Cutscenes activate when bosses are killed	N/A	Cutscenes activate when bosses are killed	N/A

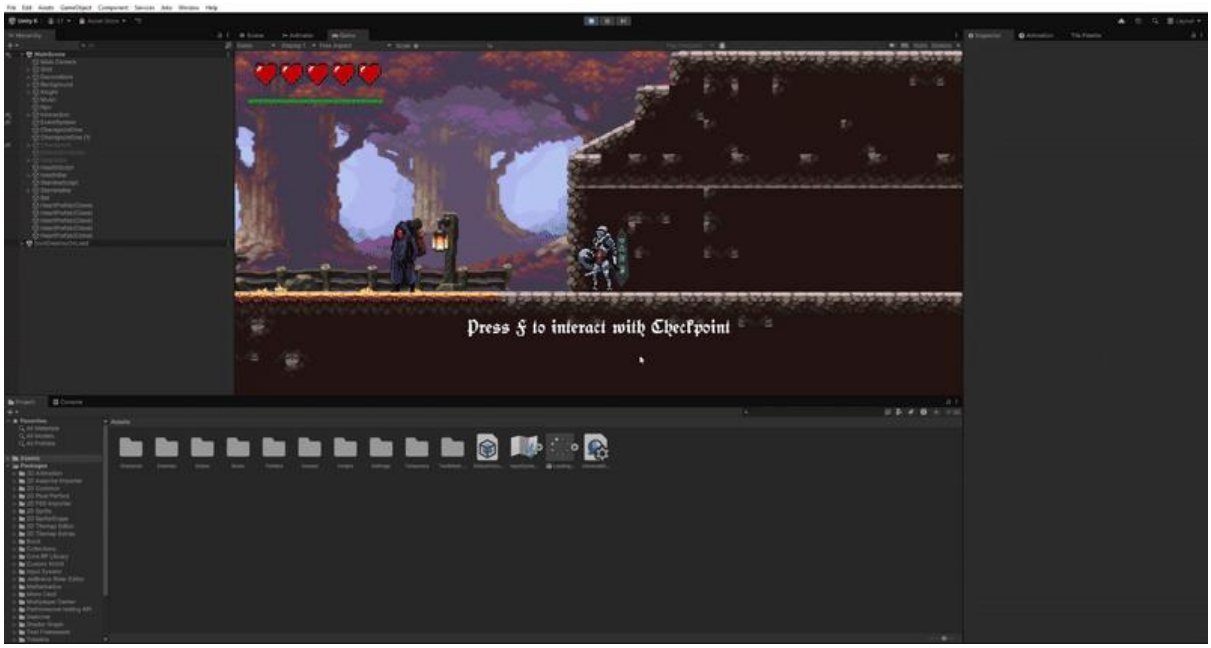
Evidence of testing



This shows how I tested the Main UI to ensure it works correctly, you can see me clicking around the screen to show that no buttons are hidden or anything will happen. I am also clicking lots of keys to show that nothing will happen and my game works as expected. The loading screen works correctly and there are no buttons on the screen. The loading screen wheel also works as expected. However, certain parts of the UI get stuck as you can see with the Control highlight got stuck as highlighted, to fix this I would have to minimize the pointer exit time for when the pointer leaves the box of the controls. Certain parts of the background images in the UI are different to the other screens, to fix this I would just scale the smaller images to be full screen size.



This shows how I tested the general gameplay to make sure it works as expected. You can see me run through each of the characters moves and then mash the keyboard to see if anything erratic happens – it did not. The stamina also went down when I was moving so it also works correctly. I then talked to the NPC to make sure his dialogue also worked as expected.



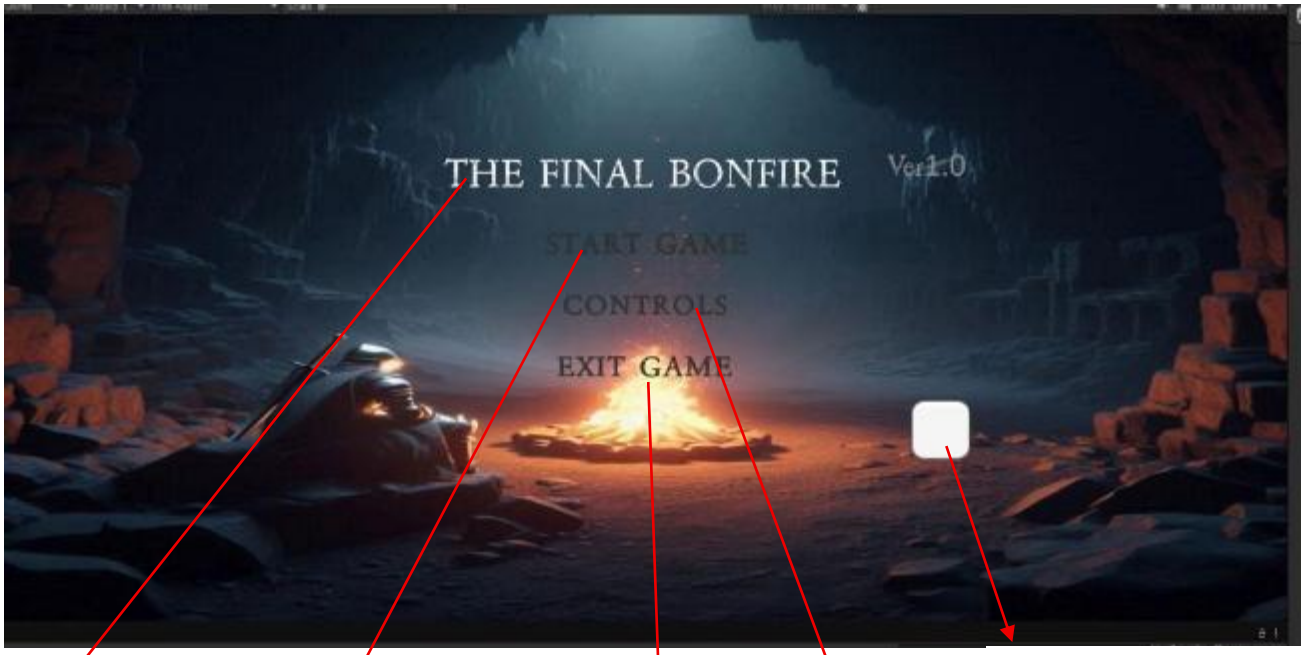
This shows how I tested more of the general gameplay as well as the checkpoint UI. I ensured that the upgrades worked as expected with health and stamina increasing when clicked. The bat also targeted me correctly and attacked me which took one of my hearts.

All of the tests I have performed have proven that my game is robust and doesn't behave unpredictably/ erratically.

Usability features

Breaking down features

Main Menu



Title of game to indicate what the user is about to play

Start game button to send the user into the game, located at the top of the screen as this will most likely be the button first clicked/ looked at

Exit game button is located at the bottom as I want people to stay playing my game. It will be he last item users will look at on the screen

Controls button is located underneath start game as it is important the user sees it to understand the controls

Sound button (speaker icon is on when music is off) this adds more usability and is a good addition to have for people that don't want to hear the music.

I can test this screen through destructive testing, allowing me to ensure there is no way for the screen to crash/ break.

Controls screen



The controls indicate what buttons the user should press to navigate through the game. It is important they understand what to do so that they can get the most out of playing the game

Title indicates what screen the user is on

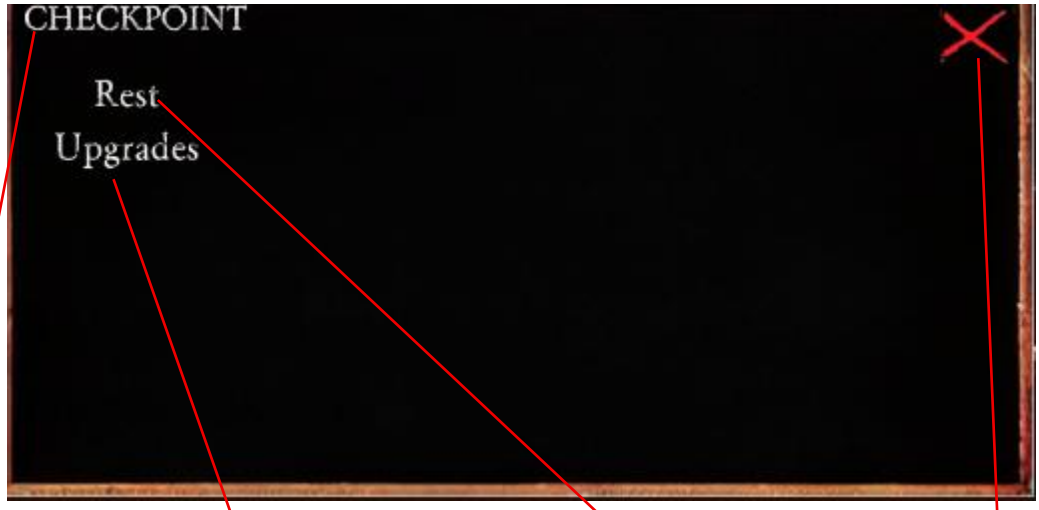
The ‘X’ button allows the user to go back to the home screen. This is needed else they would be stuck on this screen after clicking onto it

I can test this screen through destructive testing, allowing me to ensure there is no way for the screen to crash/ break.

Loading Screen



Loading wheel to indicate the game loading



The exit button allows the user to go back to playing the game

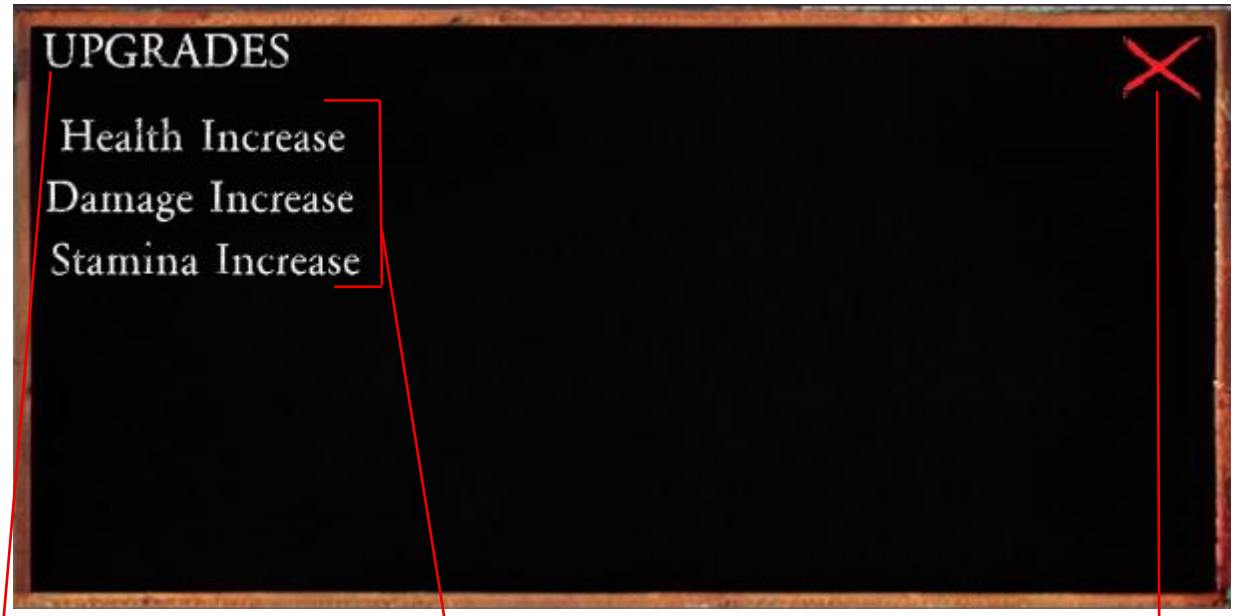
Checkpoint title to indicate what page the user is on

This button will take the user to the upgrades screen. This is needed so the user can access the place to upgrade their character

Rest button to allow the character to get back to full health (and refill flasks if it was in the game)

I can test this screen through destructive testing, allowing me to ensure there is no way for the screen to crash/ break.

Upgrade Screen



Upgrades screen title to indicate what page the user is on

These upgrades are very important for me to be able to complete success criteria point 12. It means that the user is able to upgrade their players stats so that it is easier for the player to kill enemies and promotes progression

Exit button is vital for the player to be able to exit the screen and continue with the game.

I can test this screen through destructive testing, allowing me to ensure there is no way for the screen to crash/ break.

Stakeholder testing/ review

Ed white's review of game.

User Interface

Main menu: I like the menus music; it sets the tone for a mystical fantasy game. The ability to turn the music off is a good feature. The controls screen provides a nice, simple overview of what the controls are. The menu is overall a good, straightforward screen and anything more would start to overcomplicate things.

Checkpoints: I like the checkpoints as they look good and the UI works well and is very easy to navigate. The design choice of the checkpoints is good and fits well with the futuristic yet fantasy style, showing there is other things at work. The health and stamina upgrades work well and the cap for the health is good as to not make the game too easy.

Gameplay

Movement: The movement is smooth, there is a lot of variation and the fact you can roll, as most games don't have that and is a unique aspect to the game. The animations flow well with each other and look good.

NPC: I like the NPC, it gives you can immersive feel into the story and provides a starting point of how to start the game. The words appearing at the bottom of the screen is good, however I think the words could have stayed on the screen for longer as I didn't get a chance to see all the words.

Bat: The bats animations were good, and the stationary mode of rotating back and forth when the bat is not locked onto you and woken up. It is good how the bat attacks you once close enough and follows you closely.

Overall I would say that with some extra development it could turn into a very fun game and has the ability to become a great soulslike game. All the functionality and mechanics are well done and with some more levels and enemies, as well as finishing off the characters attacks this could be very good.

Stakeholder feedback analysis

After reading the feedback from the stakeholder, I think what I have created so far is exactly what I was aiming for. I believe that the feedback provides a good overview of my game and a good insight to what I have done well and what I could improve. To fix

the problem of the NPC's dialogue not staying there long enough I could simply just make the time length of the words to stay on the screen longer.

Remaining Success Criteria

SC = Success criteria

Enemies and Bosses SC 9,15,23,24,26

I only completed the first enemy (Bat) which did take a long time to do. With extra time I would follow the same process with the other enemies (Set up animations, create code, test). However, because I have already done the bat and know the process well it would take much less time or me to make the other enemies and bosses. The only difference is the bosses would have to have some extra work put in for the status effects being added in; this would be done by adding some code that would cause the player to take damage over time or having a permanent stamina reduction. The bosses would also have to have their health bars with their names located in the middle-top of the screen so the user is able to know that it's a boss and how much health they have left. After this has been completed the extra addons in the SC would require me to enlarge the sizes of the bosses to make them visibly larger than the enemies, this would be done by increasing the scale of the bosses in the x and y axis. To add in extra attacks, I would have to repeat the process of setting up animations, creating code and then testing for each enemy. This would make the game more interesting and difficult which in turn would make the gameplay much more enjoyable.

Damage upgrade SC 12

Seeing as I have already completed the health and stamina upgrades, it would be relatively easy to add in the damage upgrade as all I have to do is update the characters overall attack damage by increasing it by a set value.

Item bar SC 13,27

To add in an item bar, given more time I would get the items first, put them around the level and then add code to them to do the desired function I want them to do. After, I would add get the item bar asset and implement it into the game, complete adding in code and run some destructive and general tests to confirm it works. This process can be repeated for adding in the extra items to nullify boss status effects.

Flames SC 14

To add flames I have to first put in a canvas, then overlay the game screen with it and put in the flames icon with a changeable number underneath. Then add some code so

that the flames will increase after an enemy is killed and extra flames are added for killing a boss. Then I will have to alter the health, stamina and damage upgrades to be dependent on if the flames value is high enough to upgrade the variable they are attempting to upgrade; the price of the upgrade will then be deducted from the total value of the flames.

Rolling invincibility SC 16

Adding roll invincibility would be relatively quick to add, I would simply just have to add in some code to dictate that when the character is rolling or while the roll animation is running the characters' health does not get affected. This could be done by either increasing the character's health a lot while the roll is happening to create the illusion that the character doesn't take any damage, when the roll is done the health would be returned to what it was.

Health flasks SC 17,18,19,20

To add health flasks, I would have to edit the health script to create a function that would allow me to increase the health of the character when I press a key (most likely H). I would then add an icon for the health flask on the canvas that the flames also feature on with a variable number underneath the flask icon. This would indicate how many flasks the user has, when the player uses one, it will decrease the number of flasks by one. After, I would alter the checkpoint script so that when the player presses the 'Rest' button it will reset the number of flasks to full. Finally, I would implement a new item that would increase the amount of flasks the player can hold at a time. This would be done through adding the upgrade asset and placing it around the levels, then I would create a subroutine that would link to the asset and make it increase the number of flasks held by 1.

Death process SC 21,22

When the character dies, a death screen needs to play out. I would do this by simply adding a new scene in, placing the words YOU DIED! In the middle of the screen and make it play for a certain amount of time. I would then add some code to the checkpoints so that when you respawn it will go to the most recently visited checkpoint. I believe that this will take quite a long time as I don't currently know how to do that and will have to have more time to research it.

Cutscenes SC 29

This will be relatively easy to do, as all I actually have to do is add in code to the boss scripts so that for example if health <= 0 { run (bossName)cutscene }.

Overall review of the project

I think overall I did well in the project in terms of efficiency of work and code quality. I could have managed my time better and granted more, I would be able to manage it a lot better and continue to finish the game. I think that the game itself was good and looks and feels like just what I was aiming for in terms of a dark fantasy, soulslike world and gameplay. There were various limitations in the making of my game, specifically time and having to manage between other a level work I had to do. I think some of the code I had written to begin with and some of the ways I had taken on certain problems could've been dealt with better so that I didn't have to waste a lot of time doing them, which could've given me more time to work on other parts of the project. If I did it again, I would be a lot more open minded to how I approach each problem and focus on the important parts of getting it right. I think the stakeholder feedback was good, and I agree completely with everything said. I would take into consideration changing the length of the NPCs speech and also finishing the characters attacks as I currently have not done that and it would make the game more like what I'm attempting to do. So therefore, with more time I am sure I would be able to turn this game into everything I have planned and much more.

Code Overview

This is all the code that was included in my game.

Character

Movement

```
using UnityEngine;

using System.Collections;

using Unity.VisualScripting;


public class Movement : MonoBehaviour

{

    //general

    private SpriteRenderer SpriteRenderer;

    public Animator animator;

    SpriteRenderer sprite;

    private Rigidbody2D rb;
```

```
    //Run,Walk
    float dirX, moveSpeed = 0f;
    bool isSprinting = false; // When true stamina will decrease


    //Jump
    public float jumpForce = 10f;
    public float jumpAngle = 45f;
    public float jumpCooldown = 1.5f;
    public float nextJumpTime = 0f;

    //Roll
    public float rollSpeed = 5f;
    public float rollCooldown = 1.5f;
    private float nextRollTime = 0f;

    //Stamina
    public StaminaSystem staminaSystem; // Reference the stamina script
    public float sprintDrain = 10f; // How much stamina per second it costs to sprint
    public float jumpCost = 15f; // How much stamina it costs to jump
    public float rollCost = 15f; // How much stamina it costs to roll
    private float lastStaminaUseTime = 0f; // When stamina was last used
    public float staminaRegenDelay = 2f;   // Delay before regen starts in seconds

    // Start is called once before the first execution of Update after the MonoBehaviour is
created
    void Start()
```

```
{
    SpriteRenderer = GetComponent<SpriteRenderer>();
    rb = GetComponent<Rigidbody2D>();
    rb.constraints = RigidbodyConstraints2D.FreezeRotation;

}

// Update is called once per frame
void Update()
{
    isSprinting = false;



    //Walk
    if (Input.GetKey(KeyCode.D))
    {
        moveSpeed = 0.5f;
        transform.Translate(Vector2.right * moveSpeed * Time.deltaTime);
        animator.SetBool("isWalking", true);
        transform.localScale = new Vector2(0.75f, 0.75f);

    }
    else if (Input.GetKey(KeyCode.A))
    {
        moveSpeed = 0.5f;
        transform.Translate(Vector2.left * moveSpeed * Time.deltaTime);
        animator.SetBool("isWalking", true);
        transform.localScale = new Vector2(-0.75f, 0.75f);
```

```
    }
    else
    {
       animator.SetBool("isWalking", false);
    }


    //Run
    if (Input.GetKey(KeyCode.LeftShift))
    {
       if(staminaSystem.currentStamina > 0f) // Checks stamina is greater than 0 to ensure character has enough stamina to perform action
       {
          lastStaminaUseTime = Time.time;
          moveSpeed = 1f;
          transform.Translate(Vector2.right * moveSpeed * Time.deltaTime);
          animator.SetBool("isRunning", true);
          transform.localScale = new Vector2(0.75f, 0.75f);
          staminaSystem.DrainStamina(sprintDrain);
          lastStaminaUseTime = Time.time;
          isSprinting = true;

       }

    }
    else if (Input.GetKey(KeyCode.LeftControl))
    {
```

```
        if(staminaSystem.currentStamina > 0f)
        {
            lastStaminaUseTime = Time.time;
            moveSpeed = 1f;
            transform.Translate(Vector2.left * moveSpeed * Time.deltaTime);
            animator.SetBool("isRunning", true);
            transform.localScale = new Vector2(-0.75f, 0.75f);
            staminaSystem.DrainStamina(sprintDrain);
            lastStaminaUseTime = Time.time;
            isSprinting = true;
        }
    }
    else
    {
        animator.SetBool("isRunning", false);


    }

    //Jump
    if(Time.time >= nextJumpTime)
    {
        if (Input.GetKeyDown(KeyCode.E))
        {
            if(staminaSystem.UseStamina(jumpCost)) // Checks stamina is greater than the stamina cost to jump
            {
                lastStaminaUseTime = Time.time;
                float angleInRadians = jumpAngle * Mathf.Deg2Rad;
```

```
            Vector2 jumpDirection = new Vector2(Mathf.Cos(angleInRadians),
Mathf.Sin(angleInRadians));
            rb.linearVelocity = jumpDirection * jumpForce;
            animator.SetTrigger("Jump");
            transform.localScale = new Vector2(0.75f, 0.75f);
            nextJumpTime = Time.time + jumpCooldown;
          }

        }
        else if (Input.GetKeyDown(KeyCode.Q))
        {
          if(staminaSystem.UseStamina(jumpCost))
          {
            lastStaminaUseTime = Time.time;
            float angleInRadians = jumpAngle * -Mathf.Deg2Rad;
            Vector2 jumpDirection = new Vector2(Mathf.Cos(angleInRadians),
Mathf.Sin(angleInRadians));
            rb.linearVelocity = -jumpDirection * jumpForce;
            animator.SetTrigger("Jump");
            transform.localScale = new Vector2(-0.75f, 0.75f);
            nextJumpTime = Time.time + jumpCooldown;
          }
        }
        else
        {
          animator.SetBool("isIdle", true);
        }
      }
```

```
    //Roll
    if (Time.time >= nextRollTime)
    {
      if (Input.GetKeyDown(KeyCode.C))
      {
        if(staminaSystem.UseStamina(rollCost)) // Checks stamina is greater than the stamina cost to roll
        {
          lastStaminaUseTime = Time.time;
          Vector2 rollDirection = transform.right;
          rb.linearVelocity = rollDirection * rollSpeed;
          animator.SetTrigger("Roll");
          transform.localScale = new Vector2(0.75f, 0.75f);
          nextRollTime = Time.time + rollCooldown;
        }
      }
      else if (Input.GetKeyDown(KeyCode.Z))
      {
        if(staminaSystem.UseStamina(rollCost))
        {
          lastStaminaUseTime = Time.time;
          Vector2 rollDirection = -transform.right;
          rb.linearVelocity = rollDirection * rollSpeed;
          animator.SetTrigger("Roll");
          transform.localScale = new Vector2(-0.75f, 0.75f);
          nextRollTime = Time.time + rollCooldown;
        }
```

```
        }
      }

      if (!isSprinting && !Input.GetKey(KeyCode.E) && !Input.GetKey(KeyCode.Q) &&
   !Input.GetKeyDown(KeyCode.C) && !Input.GetKeyDown(KeyCode.Z)) // Makes the
character regenerate if the user is not sprinting, jumping or rolling
      {
         staminaSystem.RegenerateStamina();
      }
   }
}
```

Health

```
using UnityEngine;
using UnityEngine.UI;
using System.Collections.Generic;



public class HeartHealth : MonoBehaviour
{
   public int maxHearts = 7;              // Total number of hearts (max health)
   public int currentHearts;              // Current number of active hearts

   public GameObject heartPrefab;
   public Transform heartsContainer;

   private List<GameObject> heartObjects = new(); // List to keep track of heart
GameObjects
```

```
void Start()
{
   currentHearts = maxHearts;            // Set starting health
   CreateHearts();                  // Instantiate and display hearts
}

void Update()
{
   if (Input.GetKeyDown(KeyCode.H))
   {
      TakeDamage(1);
   }
}

void CreateHearts()
{
   // Create and add heart icons to the UI
   for (int i = 0; i < maxHearts; i++)
   {
      GameObject heart = Instantiate(heartPrefab, heartsContainer);
      heartObjects.Add(heart);
   }
}

public void TakeDamage(int amount)
{
   Debug.Log("player took damage" + amount);
   currentHearts -= amount;                     // Reduce health
```

```
    currentHearts = Mathf.Clamp(currentHearts, 0, maxHearts); // Clamp to 0–max
    UpdateHearts();
}

void UpdateHearts()
{
    // Activate hearts up to current health, hide the rest
    for (int i = 0; i < heartObjects.Count; i++)
    {
        heartObjects[i].SetActive(i < currentHearts);
    }
}

public void Heal(int amount)
{
    currentHearts += amount;                    // Increase health
    currentHearts = Mathf.Clamp(currentHearts, 0, maxHearts);
    UpdateHearts();
}

public void AddHeart()
{
    if (maxHearts < 7)
    {
        maxHearts++;
        currentHearts = maxHearts;

        GameObject heart = Instantiate(heartPrefab, heartsContainer);
```

```
            heartObjects.Add(heart);

            UpdateHearts();
        }
    }

}
```

Stamina

```
using UnityEngine;
using UnityEngine.UI;

public class StaminaSystem : MonoBehaviour
{
    public float maxStamina = 100f;  // Maximum stamina the character can have
    public float currentStamina;  // The current stamina value
    public float regenRate = 5f; // Rate at which stamina regenerates per second when not in use

    public Slider staminaBar; // Reference to the UI slider for stamina bar

    void Start()
    {
        currentStamina = maxStamina; // Set current stamina to max when the game starts
        UpdateStaminaBar(); // Update UI
    }

    void Update()
```

```
{


}

public void RegenerateStamina()
{
    if (currentStamina < maxStamina)
    {
        currentStamina += regenRate * Time.deltaTime;
        currentStamina = Mathf.Min(currentStamina, maxStamina);
        UpdateStaminaBar();
    }
}

public bool UseStamina(float amount) //Subroutine for rolling, jumping and attacking
{
    if (currentStamina >= amount) // Check if there's enough stamina to perform the
action
    {
        currentStamina -= amount;
        UpdateStaminaBar();
        return true; // Action allowed
    }

    return false; // Not enough stamina to perform action
}
```

```
  public void DrainStamina(float amountPerSecond) // Subroutine to drain stamina when sprinting
  {
    currentStamina -= amountPerSecond * Time.deltaTime;
    currentStamina = Mathf.Clamp(currentStamina, 0f, maxStamina); // Don't allow stamina to go below 0
    UpdateStaminaBar(); // Refresh UI
    Debug.Log("Draining stamina. Current: " + currentStamina);
  }

  void UpdateStaminaBar() // Updates the UI stamina bar to show current value
  {
    if (staminaBar != null)
    {
      staminaBar.value = currentStamina / maxStamina;
    }
  }

  public void IncreaseMaxStamina(float amount)
  {
    maxStamina += amount; // Increases by set amount
    currentStamina = maxStamina; // Refill stamina on upgrade
    UpdateStaminaBar();

    if (staminaBar != null)
    {
      RectTransform barTransform = staminaBar.GetComponent<RectTransform>();
      if (barTransform != null)
```

```
        {
            // Increase width by 20 units
            barTransform.sizeDelta += new Vector2(20f, 0f);
        }
    }
  }
}
```

General

Side Scrolling Camera

```
using UnityEngine;

public class sideScrollingCamera : MonoBehaviour
{
  public Transform player;
  public float offsetX = 5f;
  public float smoothSpeed = 0.125f;

  // Start is called once before the first execution of Update after the MonoBehaviour is created
  void Start()
  {

  }

  // Update is called once per frame
  void LateUpdate()
  {
```

```
    Vector3 desiredPosition = new Vector3(player.position.x + offsetX,
transform.position.y, transform.position.z);
    Vector3 smoothedPosition = Vector3.Lerp(transform.position, desiredPosition,
smoothSpeed);
    transform.position = smoothedPosition;
  }
}
```

Interaction

Character interactions

```
using UnityEngine;
using TMPro;
using System.Collections;
using UnityEngine.SceneManagement;
using Unity.VisualScripting;

public class CharacterInteraction : MonoBehaviour
{
  // Reference GameObjects
  public GameObject npc;
  public GameObject[] checkpoints;
  public GameObject Checkpoint;
  public GameObject Upgrades;
  public HeartHealth heartHealth;
  public StaminaSystem staminaSystem;

  public float interactionDistance = 2.0f; // Distance within which interaction is possible
  public TextMeshProUGUI interactionText; // Reference to the TextMeshProUGUI
element
```

```
    public float displayDuration = 2.0f; // Duration to display each message
    public float letterDelay = 0.1f; // Delay between each letter

    private bool isInteracting = false; // Flag to check if interaction is ongoing



    void Start()
    {
        Checkpoint.SetActive(false);
        Upgrades.SetActive(false);
    }

    void Update()
    {
        HandleInteractions();
    }

    void HandleInteractions()
    {
        bool isInteractingWithNpc = false;
        bool isInteractingWithCheckpoint = false;



        // NPC interaction
        float distanceToNpc = Vector3.Distance(transform.position,
npc.transform.position);
        if (distanceToNpc <= interactionDistance && !isInteracting)
```

```
    {
        interactionText.text = "Press F to interact with NPC";
        isInteractingWithNpc = true;

        if (Input.GetKeyDown(KeyCode.F))
        {
            InteractWithNPC();
            isInteracting = true;  // Ensure flag is set when interacting with NPC
        }
    }


    // Checkpoint interaction
    foreach (GameObject checkpoint in checkpoints)
    {
        float distanceToCheckpoint = Vector3.Distance(transform.position,
checkpoint.transform.position);
        if (distanceToCheckpoint <= interactionDistance && !isInteracting)
        {
            interactionText.text = "Press F to interact with Checkpoint";
            isInteractingWithCheckpoint = true;
            if (Input.GetKeyDown(KeyCode.F))
            {
                CheckpointScreen(checkpoint);
                isInteracting = true; // Set interacting flag when checkpoint is activated
            }
        }
    }
```

```
    // Clear the text if no interaction
    if (!isInteractingWithNpc && !isInteractingWithCheckpoint && !isInteracting)
    {
        interactionText.text = "";
    }
}



//NPC INTERACTION
void InteractWithNPC()
{
    // Code for interaction with the NPC
    isInteracting = true; // Set the flag to true
    string[] messages = {
        "Hello fateful Harbinger of Chaos, I have waited centuries...",
        "I am here to unveil your destiny, as dearest Solaire entrusted me after claiming the Chosen Undead's flame...",
        "You must harness the final Bonfires flame to destroy this world for good, Solaire willed it so for his sun was not as it seemed...",
        "I and the Great Solaire of Astoras wish you the bestest of luck!"
    };
    StartCoroutine(DisplayMessagesCoroutine(messages));
}

IEnumerator DisplayMessagesCoroutine(string[] messages)
```

```
{
    foreach (string message in messages)
    {
        yield return StartCoroutine(DisplayMessageLetterByLetter(message));
        yield return new WaitForSeconds(displayDuration);
    }
    interactionText.text = ""; // Clear the text after all messages are displayed
    isInteracting = false; // Reset the flag
}

IEnumerator DisplayMessageLetterByLetter(string message)
{
    interactionText.text = "";
    foreach (char letter in message.ToCharArray())
    {
        interactionText.text += letter;
        yield return new WaitForSeconds(letterDelay);
    }
}

IEnumerator ResetInteractionFlag()
{
    yield return new WaitForSeconds(displayDuration);
    isInteracting = false; // Reset the flag after a delay
}



//CHECKPOINT UI
```

```
void CheckpointScreen(GameObject checkpoint)
{
   interactionText.text = "";
   Checkpoint.SetActive(true); // Make the Checkpoint screen show
   StartCoroutine(ResetInteractionFlag());



}

public void CheckpointExitButton()
{
   Checkpoint.SetActive(false);
}

public void UpgradesScreen()
{
   Upgrades.SetActive(true);
   Checkpoint.SetActive(false);
}

//UPGRADE UI
public void UpgradeExitButton()
{
   Upgrades.SetActive(false);
   Checkpoint.SetActive(true);
}

public void IncreaseHealth()
```

```
    {
        if (heartHealth.currentHearts < 7)
        {
            heartHealth.AddHeart();


        }
    }


    public void IncreaseStamina()
    {
        if(staminaSystem != null)
        {
            staminaSystem.IncreaseMaxStamina(20f); // References the
IncreaseMaxStamina subroutine from the Stamina Script and increases it by 20
        }


    }
}
```

NPC

This is the code for the interaction for just the NPC (taken from the character interactions)

```
public GameObject npc;


public float interactionDistance = 2.0f; // Distance within which interaction is possible
public TextMeshProUGUI interactionText; // Reference to the TextMeshProUGUI
element
public float displayDuration = 2.0f; // Duration to display each message
public float letterDelay = 0.1f; // Delay between each letter
void Update()
```

```
{
    HandleInteractions();
}

void HandleInteractions()
{
    bool isInteractingWithNpc = false;
private bool isInteracting = false; // Flag to check if interaction is ongoing
void Update()
{
    HandleInteractions();
}

void HandleInteractions()
{
    bool isInteractingWithNpc = false;
    bool isInteractingWithCheckpoint = false;



    // NPC interaction
    float distanceToNpc = Vector3.Distance(transform.position, npc.transform.position);
    if (distanceToNpc <= interactionDistance && !isInteracting)
    {
        interactionText.text = "Press F to interact with NPC";
        isInteractingWithNpc = true;

        if (Input.GetKeyDown(KeyCode.F))
        {
```

```
            InteractWithNPC();
            isInteracting = true;  // Ensure flag is set when interacting with NPC
        }
    }
// Clear the text if no interaction
if (!isInteractingWithNpc && !isInteractingWithCheckpoint && !isInteracting)
{
    interactionText.text = "";
}
   //NPC INTERACTION
   void InteractWithNPC()
   {
      // Code for interaction with the NPC
      isInteracting = true; // Set the flag to true
      string[] messages = {
         "Hello fateful Harbinger of Chaos, I have waited centuries...",
         "I am here to unveil your destiny, as dearest Solaire entrusted me after claiming the Chosen Undead's flame...",
         "You must harness the final Bonfires flame to destroy this world for good, Solaire willed it so for his sun was not as it seemed...",
         "I and the Great Solaire of Astoras wish you the bestest of luck!"
      };
      StartCoroutine(DisplayMessagesCoroutine(messages));
   }

   IEnumerator DisplayMessagesCoroutine(string[] messages)
   {
      foreach (string message in messages)
```

```
    {
        yield return StartCoroutine(DisplayMessageLetterByLetter(message));
        yield return new WaitForSeconds(displayDuration);
    }
    interactionText.text = ""; // Clear the text after all messages are displayed
    isInteracting = false; // Reset the flag
}

IEnumerator DisplayMessageLetterByLetter(string message)
{
    interactionText.text = "";
    foreach (char letter in message.ToCharArray())
    {
        interactionText.text += letter;
        yield return new WaitForSeconds(letterDelay);
    }
}

IEnumerator ResetInteractionFlag()
{
    yield return new WaitForSeconds(displayDuration);
    isInteracting = false; // Reset the flag after a delay
 }
```

Checkpoint

This is the code for the interaction for just the Checkpoints (taken from the character interactions)

```
public GameObject[] checkpoints;
public GameObject Checkpoint;
```

```
public float interactionDistance = 2.0f; // Distance within which interaction is possible
private bool isInteracting = false; // Flag to check if interaction is ongoing
void Start()
{
    Checkpoint.SetActive(false);
}
void Update()
{
   HandleInteractions();
}

void HandleInteractions()
{
   bool isInteractingWithCheckpoint = false;
   // Checkpoint interaction
   foreach (GameObject checkpoint in checkpoints)
   {
      float distanceToCheckpoint = Vector3.Distance(transform.position,
checkpoint.transform.position);
      if (distanceToCheckpoint <= interactionDistance && !isInteracting)
      {
         interactionText.text = "Press F to interact with Checkpoint";
         isInteractingWithCheckpoint = true;
         if (Input.GetKeyDown(KeyCode.F))
         {
            CheckpointScreen(checkpoint);
            isInteracting = true; // Set interacting flag when checkpoint is activated
         }
```

```
        }
    }


    // Clear the text if no interaction
    if (!isInteractingWithNpc && !isInteractingWithCheckpoint && !isInteracting)
    {
        interactionText.text = "";
    }
}
//CHECKPOINT UI
void CheckpointScreen(GameObject checkpoint)
{
    interactionText.text = "";
    Checkpoint.SetActive(true); // Make the Checkpoint screen show
    StartCoroutine(ResetInteractionFlag());



}

public void CheckpointExitButton()
{
    Checkpoint.SetActive(false);
}

public void UpgradesScreen()
{
    Upgrades.SetActive(true);
```

```
    Checkpoint.SetActive(false);
}
```

Upgrades

This is the code for the interaction for just the Upgrades (taken from the character interactions)

```
//UPGRADE UI
public void UpgradeExitButton()
{
    Upgrades.SetActive(false);
    Checkpoint.SetActive(true);
}

public void IncreaseHealth()
{
    if (heartHealth.currentHearts < 7)
    {
        heartHealth.AddHeart();

    }
}

public void IncreaseStamina()
{
    if(staminaSystem != null)
    {
        staminaSystem.IncreaseMaxStamina(20f); // References the IncreaseMaxStamina subroutine from the Stamina Script and increases it by 20
    }
```

}

Tests

TextMeshPro calling test

This was used to ensure that I was calling the TMP object built into unity correctly as I was having trouble with it when making the NPC interaction for the words showing up on the bottom of the screen.

```
using UnityEngine;

using TMPro;



public class texttestscript : MonoBehaviour

{

  public TextMeshProUGUI testText;


  // Start is called once before the first execution of Update after the MonoBehaviour is created

  void Start()

  {

    testText.text = "Hello, World!";

  }


  // Update is called once per frame

  void Update()

  {


  }
```

UI

Loading Screen

```
using UnityEngine;
using UnityEngine.SceneManagement;
using System.Collections;

public class loadingScreen1 : MonoBehaviour
{
    public Animator animator;

    // Start is called once before the first execution of Update after the MonoBehaviour is created
    void Start()
    {
        StartCoroutine(TransitionToNextScene());
    }

    // Update is called once per frame
    void Update()
    {

    }

    IEnumerator TransitionToNextScene()
    {
        yield return new WaitForSeconds(5f);
        SceneManager.LoadScene(2);
    }
```

```
}
```

Main Menu

```
using Unity.VisualScripting;
using UnityEditor;
using UnityEngine;
using UnityEngine.SceneManagement;



public class mainMenu : MonoBehaviour
{
    //reference gameobjects (Both Canvases)
    public GameObject MainMenuCanvas;
    public GameObject ControlsScreen;



    // Start is called once before the first execution of Update after the MonoBehaviour is created
    void Start()
    {
        MainMenuCanvas.SetActive(true);
        ControlsScreen.SetActive(false);
    }

    // Update is called once per frame
    void Update()
    {
```

```
    }

    public void StartGame()
    {
        SceneManager.LoadScene(1);
    }

    public void ExitGame()
    {
        EditorApplication.isPlaying = false;
    }

    public void Controls() // show the controls screen and turn off the main menu screen
    {
        ControlsScreen.SetActive(true);
        MainMenuCanvas.SetActive(false);
    }

    public void ExitButton() // show the main menu screen and turn off the control screen
    {
        ControlsScreen.SetActive(false);
        MainMenuCanvas.SetActive(true);
    }
}
```